Learn++: A Classifier Independent Incremental Learning Algorithm for Supervised Neural Networks

Robi Polikar, Jeff Byorick, Stefan Krause, Anthony Marino and Michael Moreton
Electrical and Computer Engineering, Rowan University,
136 Rowan Hall, Glassboro, NJ 08028, USA.

Abstract – A versatile incremental learning algorithm is introduced for supervised neural network type classifiers. The proposed algorithm, called Learn++, exploits the synergistic expressive power of an ensemble of weak classifiers for learning additional information from new data. Learn++ is capable of learning new classes, without forgetting previously acquired knowledge, even when the previously used data is no longer available. Furthermore, Learn++ is independent of the specific type of the classifier, and adds the incremental learning capability to any supervised neural network classifier.

I. INTRODUCTION

A. Incremental Learning

Supervised neural networks have enjoyed a remarkable success as effective and practical tools for a broad range of pattern recognition and function approximation applications. As with any type of classifier, the performance of neural networks relies heavily on the availability of a representative set of training examples. In many practical applications, however, acquisition of such a representative dataset is expensive and time consuming. Consequently, such data often become available in small and separate batches at different times. In such settings, it is necessary to incrementally update an existing classifier to accommodate new data while retaining the information obtained from old data.

Many of the commonly used classifiers, including the multilayer perceptron (MLP), radial basis function (RBF) networks, wavelet networks, and probabilistic neural networks (PNN), however, are unable to learn incrementally from new data. Traditionally, when new data become available, such classifiers are discarded and retrained with the composite data obtained by combining all the data accumulated thus far. This approach, however, results in loss of all previously acquired information, commonly known as *catastrophic forgetting*. Furthermore, this approach may not even be feasible if the original dataset is no longer available.

It should be noted that learning new information incrementally, without forgetting any previously acquired knowledge is not possible, unless all of the data are memorized (as in instance base classifiers, such as nearest neighbor classifiers). This phenomenon is dictated by the stability-plasticity dilemma [1]: some information may have to be lost to learn new information, as learning new patterns will tend to overwrite formerly acquired knowledge. The dilemma points out the fact that a completely stable classifier will preserve existing knowledge, but will not accommodate any new information, whereas a completely plastic classifier will learn new information but will not conserve prior knowledge.

Various algorithms with various interpretations of the phrase "incremental learning" have been suggested in the literature, all of which fall somewhere along the stability-plasticity spectrum. For instance, in some cases "incremental learning" has been used to refer to growing or pruning classifier architectures [2], or to selection of most informative training samples [3]. In other cases, some form of controlled modification of classifier weights have been suggested, typically by retraining with misclassified instances [4, 5]. Other incremental learning algorithms, such as incremental construction of support vector machines [6], incremental learning based on reproducible kernel Hilbert spaces [7], or incrementally adding new IF-THEN rules to an existing fuzzy inference system [8] have also been suggested.

For the purposes of this paper, we define an incremental learning algorithm as a classifier that

i. can learn additional information from new data;
ii. does not require access to previously used data;
iii. retains a substantial majority of the previously acquired knowledge when learning new information;
iv. can learn instances of previously unseen classes;
v. is versatile enough to be used with a variety of classification algorithms.

Algorithms referenced above, as well as many others, are all capable of learning new information; however, they do not simultaneously satisfy all of the above-mentioned criteria for incremental learning: they either require old data, forget substantial amount of prior knowledge along the way, or cannot accommodate new classes. One prominent exception is the (fuzzy) ARTMAP algorithm [9], which is indeed capable of learning new information, even when new classes are introduced. Therefore, with the exception of the fifth criterion, ARTMAP fits perfectly into our description of incremental learning. ARTMAP, however, has its own drawbacks. It has been reported in numerous occasions that ARTMAP is very sensitive to selection of its vigilance parameter, to noise levels in the training data and to the order in which the training data are presented to the algorithm. Furthermore, the algorithm often suffers from overfitting problems, resulting in poor generalization performance, if the vigilance parameter is not chosen appropriately. Various approaches have been suggested to overcome one or more of these difficulties [10, 11, 12, 13, 14, 15], though, none addressing all problems simultaneously.

The purpose of this study was therefore developing an incremental learning algorithm satisfying a rather strict set of five criteria, while avoiding the aforementioned drawbacks.

0-7803-7278-6/02/$10.00 ©2002 IEEE

B. Ensemble of Classifiers

Learn++, an algorithm based on ensemble of classifiers, is proposed in this paper as an alternate approach to incremental learning. Learn++ was inspired by the AdaBoost *(adaptive boosting)* algorithm [16], originally developed to improve the classification performance of weak classifiers. In essence, both Learn++ and AdaBoost generate an *ensemble of weak classifiers*, which are trained using different distributions of training samples. The outputs of these classifiers are then combined using the majority-voting scheme [17] to obtain the final classification rule. The approach takes advantage of the so-called *instability* of the weak classifiers, which can construct sufficiently different decision surfaces for minor modifications in their training datasets.

Using ensemble of classifiers for improving classifier accuracy has been well researched, however, its potential for addressing the incremental learning problem has not been studied in detail. There have been some attempts for using hierarchical mixture of classifiers (HMEs) in an online setting to incrementally learn from incoming data [18], however such attempts have not addressed all of the above mentioned issues of incremental learning, in particular learning new classes. This leads us to consider other adaptations of ensemble-based methods to achieve incremental learning.

Learn++ was first introduced recently in [19], as an incremental learning algorithm for MLP networks. In the next section, an improved and more versatile version of the Learn++ algorithm is introduced, followed by simulation results on various real world problems. In particular, we show that Learn++ can add the capability of incremental learning to *any* supervised neural network classifier. We also compare Learn++ and fuzzy ARTMAP both from performance and parameter selection point of view. Finally, we summarize our conclusions and point at future research directions.

II. LEARN++

Learn++ exploits the synergistic expressive power of an ensemble of classifiers to incrementally learn additional information from new data. Specifically, for each database that becomes available, Learn++ generates a number of relatively weak classifiers, whose outputs are combined through weighted majority voting to obtain the final classification. The weak classifiers, also referred to as weak hypotheses, are generated using strategically chosen subsets of the currently available database. A dynamically updated distribution over the training data instances is computed for this purpose, where the distribution is biased towards those instances that have not been properly learned by the previous hypotheses. The Learn++ algorithm is provided in Figure 1.

For each database $\mathcal{D}_k$, $k=1,\ldots,K$ that becomes available to Learn++, the inputs to the algorithm are (i) a sequence of m training data instances x_i along with their correct labels y_i, (ii) a weak classification algorithm **WeakLearn** to be used as a base classifier, and (iii) an integer T_k specifying the number of classifiers (hypotheses) to be generated for that database. The WeakLearn can be any supervised classifier that can correctly classify at least half of the instances of its own training dataset. Note that this is not a very restrictive requirement. In fact, for a two-class problem, it is the least we can expect from a classifier. As mentioned above, the emphasis on the weakness is to allow sufficiently different decision boundaries to be generated with slightly different training datasets. The weak classifiers also have the added advantage of rapid training: unlike stronger classifiers, they generate only a rough approximation of the decision boundary, avoiding significant training time otherwise spent on fine-tuning the decision boundary. This further helps in preventing overfitting, a much welcome asset to any classifier.

A neural network whose parameters are chosen appropriately can serve as a weak classifier. In particular, by keeping the network size small and the error goal high, with respect to the complexity of the classification problem, a neural network can emulate a weak classifier.

Learn++ starts by initializing a weight distribution, according to which a training subset TR_t and a test subset TE_t are drawn at the t^{th} iteration of the algorithm. Unless there is prior reason to choose otherwise, this distribution is initially set to be uniform, giving equal probability to each instance to be selected into the first training subset. When an additional database is introduced, the weights are adjusted by the **Init_dist** subroutine as described later in this section.

At each iteration t, the weights w_t from previous iteration are first normalized to ensure that a legitimate distribution D_t is obtained (step 1). Training and test subsets are then selected according to D_t (step 2), and the weak classifier is trained with the training subset (step 3). A hypothesis h_t is obtained as the t^{th} classifier, whose error is then computed on $TR_t + TE_t$ (entire current database $\mathcal{D}_k$) simply by adding the distribution weights of the misclassified instances

$$\varepsilon_t = \sum_{i:h_t(x_i)\neq y_i} D_t(i). \qquad (1)$$

The WeakLearn requirement for classifying at least half of the training data is enforced by requiring that the error, as defined in Equation (1), be less than ½. If this is the case, the error is normalized to obtain the normalized error

$$\beta_t = \varepsilon_t / (1-\varepsilon_t), \qquad 0 < \beta_t < 1. \qquad (2)$$

If the error is not less than half, then the current hypothesis is discarded, and a new training subset is selected (step 4). All hypotheses generated thus far are then combined using the weighted majority voting to obtain the composite hypothesis H_t (step 5).

$$H_t = \arg\max_{y\in Y} \sum_{t:h_t(x)=y} \log(1/\beta_t) \qquad (3)$$

The weighted majority voting simply chooses the class receiving the highest vote from all hypotheses. The voting is less then democratic, however, since each hypothesis has a voting weight that is inversely proportional to its normalized error. Those hypotheses with good performance records are awarded higher voting weights.

0-7803-7278-6/02/$10.00 ©2002 IEEE

Algorithm Learn++

Input: For each dataset drawn from $\mathcal{D}_k$ $k=1,2,\ldots,K$

- Sequence of m examples $S=[(x_1,y_1),(x_2,y_2),\ldots,(x_m,y_m)]$.
- Weak learning algorithm **WeakLearn**.
- Integer T_k, specifying the number of iterations.

Do for each $k=1,2,\ldots,K$:

Initialize $w_1(i) = D_1(i) = 1/m,\ \forall i,\ i = 1,2,\cdots,m$

Call Init_dist if k>1 for distribution initialization when a new database becomes available

Do for $t = 1,2,\ldots,T_k$:

1. Set $D_t = \mathbf{w}_t \Big/ \sum_{i=1}^{m} w_t(i)$ so that D_t is a distribution.
2. Draw training TR_t and testing TE_t subsets from D_t.
3. Call **WeakLearn** to be trained with TR_t.
4. Obtain a hypothesis $h_t : X \rightarrow Y$, and calculate the error of h_t : $\varepsilon_t = \sum_{i:h_t(x_i)\neq y_i} D_t(i)$ on $TR_t + TE_t$.

   If $\varepsilon_t > ½$, discard h_t and go to step 2. Otherwise, compute normalized error as $\beta_t=\varepsilon_t / (1-\varepsilon_t)$.
5. Call weighted majority voting and obtain the composite hypothesis
$$H_t = \arg\max_{y\in Y} \sum_{t:h_t(x)=y} \log(1/\beta_t)$$
6. Compute the error of the composite hypothesis
$$E_t = \sum_{i:H_t(x_i)\neq y_i} D_t(i) = \sum_{i=1}^{m} D_t(i)\left[\left[H_t(x_i) \neq y_i \right]\right]$$
7. Set $B_t = E_t/(1-E_t)$, and update the weights:
$$w_{t+1}(i) = w_t(i) \times \begin{cases} B_t, & \text{if } H_t(x_i) = y_i \\ 1, & \text{otherwise} \end{cases}$$
$$= w_t(i) \times B_t^{\,1-\left[\left[H_t(x_i)\neq y_i\right]\right]}$$

Call Weighted majority voting and
Output the final hypothesis:
$$H_{final}(x) = \arg\max_{y\in Y} \sum_{k=1}^{K} \sum_{t:h_t(x)=y} \log\frac{1}{\beta_t}$$

Figure 1. Algorithm Learn++

The error of the composite hypothesis is then computed in a similar fashion as the sum of distribution weights of the instances that are misclassified by H_t (step 6)

$$E_t = \sum_{i:H_t(x_i)\neq y_i} D_t(i) = \sum_{i=1}^{m} D_t(i)\left[\left[H_t(x_i) \neq y_i \right]\right] \quad (4)$$

where $\left[\left[\bullet \right]\right]$ evaluates to 1, if the predicate holds true. The normalized composite error B_t is then computed as

$$B_t = E_t/(1-E_t) \ , \quad 0 < B_t < 1 \quad (5)$$

which is then used in updating instance weights (step 7) :

$$w_{t+1}(i) = w_t(i) \times \begin{cases} B_t, & \text{if } H_t(x_i) = y_i \\ 1, & \text{otherwise} \end{cases} \quad (6)$$
$$= w_t(i) \times B_t^{\,1-\left[\left[H_t(x_i)\neq y_i\right]\right]}$$

The weight update rule in Equation (6) reduces the weights of those instances that are correctly classified, so that their probability of being selected into the next training subset is reduced. The probability of misclassified instances being selected into the next training subset, are therefore effectively increased. In essence, the algorithm is forced to focus more and more on instances that are difficult to classify, or in other words, instances that have not yet been properly learned. In an incremental learning setting, the newly introduced instances, particularly those coming from previously unseen classes, are precisely the instances that have not been learned yet, and therefore the algorithm quickly focuses on these instances. When a new database is introduced, the **Init_dist** routine initializes the weight distribution for a smooth transition. This routine simply jumps to step 5 and runs the rest of the **do loop** one additional time, that is, it calls the weighted majority – with the new database –, determines which instances are misclassified by the current composite hypothesis, and initializes the weight distribution accordingly. This works quite effectively, particularly when instances from a previously unknown class are introduced. At any point, a final hypothesis H_{final} can be obtained by combining all hypotheses that have been generated thus far using the weighted majority voting rule

$$H_{final}(x) = \arg\max_{y\in Y} \sum_{k=1}^{K} \sum_{t:h_t(x)=y} \log\frac{1}{\beta_t}. \quad (7)$$

It is important to emphasize that the weight update rule is the heart of the Learn++ algorithm, as it allows effective acquisition of novel information, when new databases are introduced. The weight update rule, based on the performance of composite hypothesis, is one of the most important distinctions of Learn++ as compared to other ensemble approaches such as the AdaBoost.

III. LEARN++ SIMULATION RESULTS

Learn++ has been tested on a diverse set of synthetic and real world classification problems, including incremental learning with or without new classes being introduced. Simulation results on three such real world problems requiring incremental learning of new classes are presented here. Results on other databases, including those that do not introduce new classes, as well as detailed information regarding the databases and sample patterns are provided on the web [20].

Furthermore, we also show that Learn++ works with a variety of supervised neural networks, such as MLP, RBF and PNN, used as base (weak) classifiers. The weakness of

these classifiers – with respect to the difficulty of the classification problem – was controlled by restricting network sizes, but more importantly, by early termination of training, enforced by a relatively high error goal. Typical classification performance of an individual hypothesis on its own training data was kept relatively low to ensure sufficient weakness.

A. Ultrasonic Weld Inspection (UWI) Database

This rather challenging database was obtained by ultrasonic scanning of welding regions of nuclear power plant pipings and tubings. The welding regions, also known as heat-affected zones, are highly susceptible to growing a variety of defects, including potentially dangerous cracks. The discontinuities within the material, such as the air gaps due to cracks, cause the ultrasonic wave to be reflected back, to be received by the transducer. The reflected ultrasonic wave, also called the A-scan, serves as the signature pattern of the discontinuity, which is then analyzed to determine whether it was originated from a crack. This analysis, however, is hampered by the fact that there are other types of discontinuities, such as porosity (POR), slag and lack of fusion (LOF), all of which generate very similar A-scans, creating highly overlapping patterns in the feature space.

The four-class, 150-feature database was divided into three training datasets, $S_1 \sim S_3$, and a test set, *TEST*, used for validation. Table 1 presents the data distribution in each dataset. Each subsequent dataset was designed to introduce additional classes: S_1 had instances from slag and LOF, S_2 introduced crack instances and S_3 introduced porosity instances. *TEST* dataset included instances from all classes. Only S_k was used during the k^{th} training session $\mathbf{TS_k}$. Therefore, previously used data was not made available to Learn++ for training in future sessions. Apart from generalization performance on *TEST* dataset, the classification performance on all current and previously seen training datasets were also computed *after* each training session. These performance figures are presented in Tables 2, 3 and 4 where PNN, RBF and MLP type networks were used as base classifiers, respectively. In each case, the classification performances on the training datasets are shown in rows labeled by S_1, S_2, and S_3, as obtained after each training session, shown in columns. The numbers in parentheses indicate the number of weak classifiers generated in each training session. Classifier generation continued until the generalization performance on the *TEST* dataset, shown in the last row, reached a steady saturation value. Several observations can be made from these tables:

(i) steadily increasing generalization performances indicate that Learn++ was able to learn new information and new classes as they became available;

(ii) there is indeed a decline on training data performances indicating some loss, albeit minor, of previously acquired knowledge as new information is acquired;

(iii) Learn++ is indeed classifier independent, adding incremental learning capability to any supervised network;

Table 1.Data distribution for the UWI database

↓Dataset	LOF	SLAG	CRACK	POR
S_1	300	300	0	0
S_2	200	200	200	0
S_3	150	150	137	99
TEST	**200**	**200**	**150**	**125**

Table 2. Learn++ performance on UWI data with PNN

↓ Dataset	TS_1 (7)	TS_2 (7)	TS_3 (14)
S_1	99.5%	83.6%	71.5%
S_2	---	93.0%	75.0%
S_3	---	---	99.4%
TEST	**48.9%**	**62.4%**	**76.1%**

Table 3. Learn++ performance on UWI data with RBF

↓ Dataset	TS_1 (9)	TS_2 (5)	TS_3 (14)
S_1	90.8%	78.3%	69.8%
S_2	---	89.5%	75.8%
S_3	---	---	94.7%
TEST	**47.7%**	**60.9%**	**76.4%**

Table 4. Learn++ performance on UWI data with MLP

↓Dataset	TS_1 (8)	TS_2 (3)	TS_3 (17)
S_1	98.2%	84.7%	80.3%
S_2	---	98.7%	86.5%
S_3	---	---	97.0%
TEST	**48.9%**	**66.8%**	**78.4%**

(iv) near 50% generalization performance after TS_1 makes intuitive sense, since at that time the algorithm had only seen two of the four classes that appear in the *TEST* data. The performance gradually increases, proportional to the ratio of the classes seen, as they become available.

B. Gas Identification (GI) Database

The gas identification (GI) database used in this study consisted of responses of six quartz crystal microbalances to five volatile organic compounds, including ethanol (ET), xylene (XL), octane (OC), toluene (TL), and trichloroethelene (TCE). The database included 384 six-dimensional signals, half of which were used for training. The database was divided into three training datasets $S_1 \sim S_3$ and one validation dataset, *TEST*. The data distribution is shown in Table 5, which was specifically biased towards the new class: database S_1 had instances from ET, OC and TL, S_2 added instances mainly from TCE (and very few from the previous three), and S_3 added instances from XL (and very few from the previous four). *TEST* set included instances from all classes. As always, only S_k was used during the k^{th} training session TS_k. Tables 6, 7 and 8 present Learn++ classification performances with PNN, RBF and MLP type networks used as base classifiers, respectively.

Table 5.Data distribution for the GI database

↓Dataset	ET	OC	TL	TCE	XL
S_1	20	20	40	0	0
S_2	5	5	5	25	0
S_3	5	5	5	5	40
TEST	**34**	**34**	**62**	**34**	**40**

Table 6. Learn++ performance on GI data with PNN

↓Dataset	TS_1 (2)	TS_2 (3)	TS_3 (3)
S_1	93.7%	73.7%	76.3%
S_2	---	92.5%	82.5%
S_3	---	---	85.0%
TEST	**58.8%**	**67.7%**	**83.8%**

Table 7. Learn++ performance on GI data with RBF

↓Dataset	TS_1 (5)	TS_2 (6)	TS_3 (7)
S_1	97.5%	81.2%	76.2%
S_2	---	97.0%	95.0%
S_3	---	---	90.0%
TEST	**59.3%**	**67.6%**	**86.3%**

Table 8. Learn++ performance on GI data with MLP

↓Dataset	TS_1 (10)	TS_2 (5)	TS_3 (9)
S_1	87.5%	75.0%	71.3%
S_2	---	82.5%	85.0%
S_3	---	---	86.7%
TEST	**58.8%**	**71.1%**	**87.2%**

Performance figures listed above follow a similar trend to those on the UWI database. The steady increase on the generalization performance indicate that Learn++ was able to incrementally learn additional information provided by the new classes, regardless of the base classifier.

As a comparison, Table 9 presents the generalization performances of fuzzy ARTMAP for three different values of the vigilance parameter ρ. ARTMAP was also trained incrementally with identical training databases S_1~S_3 and tested on the identical *TEST* dataset.

Table 9. Fuzzy ARTMAP performance on GI data

↓Dataset	TS_1	TS_2	TS_3
TEST (ρ=0.85)	54.9%	68.1%	82.8%
TEST (ρ=0.90)	50.5%	67.2%	83.8%
TEST (ρ=0.95)	43.6%	59.3%	71.1%

Learn++ performs comparable to, or better then, the ARTMAP, with the added advantage of being relatively insensitive to the order of data presentation and variations in its internal parameters, including the error goal, number of hidden layers, percentage of data used as a training subset, and even to a great extend, the choice of the base classifier.

C. Optical Character Recognition (OCR) Database

This benchmark database, obtained from the UCI machine learning repository [21], consisted of 3823 training instances and 1797 test instances of digitized hand written characters. The characters were digits, 0 through 9, digitized on an 8x8 grid, creating 64 attributes for 10 classes. Only a subset of the training data was used as shown in Table 10, where the datasets are formatted in columns due to large number of classes. Note from Table 6 that multiple new classes were introduced with new databases. In particular, S_1 consisted of classes 0, 2, 4, 6, and 8, S_2 added classes 1, 3, 5 and 9, and S_3 added class 7, whereas certain previously available classes were removed in each of the subsequent databases. Also, note that S_4 did not introduce any new classes.

Table 10. Data distribution for the OCR database

↓ Class	S_1	S_2	S_3	S_4	*TEST*
0	100	50	50	25	**178**
1	0	150	50	0	**182**
2	100	50	50	25	**177**
3	0	150	50	25	**183**
4	100	50	50	0	**181**
5	0	150	50	25	**182**
6	100	50	0	100	**181**
7	0	0	150	50	**179**
8	100	0	0	150	**174**
9	0	50	100	50	**180**

Table 11. Learn++ performance on OCR data with PNN

↓Dataset	TS_1 (2)	TS_2 (5)	TS_3 (6)	TS_4 (3)
S_1	99.8%	80.8%	78.2%	92.4%
S_2	---	96.3%	89.0%	88.7%
S_3	---	---	98.2%	94.0%
S_4	---	---	---	88.0%
TEST	**48.7%**	**69.1%**	**82.8%**	**86.7%**

Table 12. Learn++ performance on OCR data with RBF

↓Dataset	TS_1 (4)	TS_2 (6)	TS_3 (8)	TS_4 (15)
S_1	98.0%	81.0%	77.0%	93.4%
S_2	---	96.5%	88.4%	80.6%
S_3	---	---	93.6%	92.7%
S_4	---	---	---	90.0%
TEST	**47.8%**	**73.2%**	**79.8%**	**85.9%**

Table 13. Learn++ performance on OCR data with MLP

↓Dataset	TS_1 (18)	TS_2 (30)	TS_3 (23)	TS_4 (3)
S_1	96.6%	89.8%	86.0%	94.8%
S_2	---	87.1%	89.4%	87.9%
S_3	---	---	92.0%	92.2%
S_4	---	---	---	87.3%
TEST	**46.6%**	**68.9%**	**82.0%**	**87.0%**

Tables 11, 12 and 13 summarize the classification performances of Learn++ on the OCR data with PNN, RBF and MLP used as base classifiers, respectively. Similar to previous cases, the results indicate that Learn++ was again able to incrementally learn new information, even when multiple new classes are presented in subsequent databases.

Recall from Table 10 that no new classes were introduced with S_4, which manifests itself in the generalization performance after TS_4. After this training session, there was only a modest improvement in the generalization performance, which can be explained by the fact that majority of the information to be learned was already learned by previous networks, and that S_4 provided only a marginal amount of new information. We also note that there was virtually no loss of prior knowledge after TS_4, which indicates that loss of prior knowledge is due to aggressive focusing on new classes, when they are present. Therefore, if and whenever possible, adding a new dataset that includes instances from previously seen classes, would be beneficial in relearning any information that might have been lost during previous training sessions.

IV. DISCUSSIONS & CONCLUSIONS

Learn++ is introduced as a classifier independent algorithm that adds the incremental learning capability to supervised neural networks. Learn++ takes advantage of the synergistic expressive power of an ensemble of weak classifiers. Each classifier is trained with a training data subset drawn from a carefully tailored distribution of the database. Individual classifiers are then combined using the weighted majority-voting rule to obtain the final classifier.

Learn++ is capable of learning new information provided by subsequent databases, including new knowledge provided by instances of previously unseen classes. Furthermore, unlike most other supervised classifiers, Learn++ does not suffer from catastrophic forgetting. Some knowledge is indeed forgotten while new information is being learned, however, this appears to be negligible, as indicated by the steady improvement in the generalization performance of the algorithm. Learn++ has been tested on three real world databases, with three different supervised networks used as base classifiers. The results demonstrate that Learn++ works rather well in a variety of practical applications.

Through out the simulations, a number of additional observations were made, not readily apparent from the tables. In particular, we have tested the sensitivity of Learn++ to the order of presentation of the data, as well as to minor changes in its parameters. We have found out that the classification performance of Learn++ was virtually the same, regardless of the order in which databases (and therefore the classes) were presented to the algorithm. Furthermore, the algorithm was considerably robust to changes to its internal parameters, such as the network size (for MLP), spread constant (for RBF and PNN), error goal, the number of hypotheses generated, and even to the type of base classifier.

One might also wonder, what generalization performance could be achieved, if the entire database were available for strong learning. To answer this question, we trained strong classifiers on entire training databases, for each case. The results were all in the middle 80% to lower 90% range, similar to, or only slightly better than those of Learn++ were. This rather satisfying result further indicates the feasibility of Learn++ as an alternative to other incremental learning algorithms, as well as to non-incremental strong classifiers.

Future work on Learn++ includes investigating various optimization opportunities for the distribution update rule and the voting mechanism, two issues constituting the heart of the algorithm. Learn++ will also be tested on function approximation type problems, as well as with other types of base classifiers. Finally, the voting mechanism hints a simple procedure for estimating the confidence of the algorithm on its own decision. If a vast majority of the hypotheses agree on the class of an unknown instance, this can be interpreted as a high confidence decision, whereas a mere majority of votes may indicate a low confidence decision. Whether this intuitive approach agrees with performance confidence intervals, as determined by a statistical analysis of the algorithm outcome, is also a topic for future research.

V. REFERENCES

[1] S. Grossberg, "Nonlinear neural networks: principles, mechanisms and architectures," Neural Networks, vol. 1, no. 1, pp. 17-61, 1988.

[2] B. Zhang, "An incremental learning algorithm that optimizes network size and sample size in one trial," Proc. of IEEE Int. Conf. on Neural Networks, pp. 215-220, 1994.

[3] A.P. Engelbrecht and R. Brits, "A clustering approach to incremental learning for feedforward neural networks," Proc. of Int. Joint Conf. on Neural Networks, vol. 3, pp. 2019-2024, 2001.

[4] L. Fu, H.H. Hsu, and J.C. Principe, "Incremental backpropagation learning networks," IEEE Trans. on Neural Networks, vol. 7, no. 3, pp. 757-762, 1996.

[5] L. Grippo, "Convergent on-line algorithms for supervised learning in neural networks," IEEE Trans. on Neural Networks, vol. 11, no. 6, pp. 1284-1299, 2000.

[6] A. Shilton, M. Palaniswami, D. Ralph, A. C. Tsoi, Incremental training of support vector machines, Proc. of Int. Joint Conf. on Neural Networks, vol.2, pp. 1197-1202, 2001

[7] S. Vijayakumar and H. Ogawa, "RKHS-based functional analysis for exact incremental learning," Neurocomputing, vol. 29, pp. 85-113, 1999.

[8] G. G. Yen and P. Meesad, "An effective neuro-fuzzy paradigm for machine condition health monitoring," IEEE Trans. on Systems, Man and Cybernetics, Part B, vol. 31, no. 4, pp. 523-536, 2001.

[9] G.A. Carpenter, S. Grossberg, N. Markuzon, J.H. Reynolds, and D.B. Rosen, "Fuzzy ARTMAP: A neural network architecture for incremental supervised learning of analog multidimensional maps," IEEE Trans. on Neural Networks, vol. 3, no. 5, pp. 698-713, 1992.

[10] J.R. Williamson, "Gaussian ARTMAP: a neural network for fast incremental learning of noisy multidimensional maps," Neural Networks, vol. 9, no. 5, pp. 881-897, 1996.

[11] C.P. Lim and R.F. Harrison, "An incremental adaptive network for on-line supervised learning and probability estimation," Neural Networks, vol. 10, no. 5, pp. 925-939, 1997.

[12] F. H. Hamker, "Life-long learning cell structures – continuously learning without catastrophic interference," Neural Networks, vol. 14, no. 4-5, pp. 551-573, 2000.

[13] G.C. Anagnostopoulos and M. Georgiopoulos, "Ellipsoid ART and ARTMAP for incremental clustering and classification," Proc. of Int. Joint Conf. on Neural Networks, vol. 2, pp. 1221-1226, 2001.

[14] S.J. Verzi, G.L. Heileman, M. Georgiopoulos, and M.J. Healy, "Rademacher penalization applied to fuzzy ARTMAP and boosted ARTMAP," Proc. of Int. Joint Conf. on Neural Networks, vol.2, pp. 1191-1196, 2001.

[15] E. Gomez-Sanchez, Y.A. Dimitriadis, J.M. Cano-Izquierdo, J. Lopez-Coronado, Safe-μARTMAP: A new solution for reducing category proliferation in Fuzzy ARTMAP, Proc. of Int. Joint Conf. on Neural Networks, vol.2, pp. 1197-1202, 2001.

[16] Y. Freund and R. Schapire, "A decision theoretic generalization of on-line learning and an application to boosting," Computer and System Sciences, vol. 57, no. 1, pp. 119-139, 1997.

[17] N. Littlestone and M. Warmuth, "Weighted majority algorithm, " Information and Computation, vol. 108, pp. 212-261, 1994.

[18] C.K. Tham, "On-line learning using hierarchical mixtures of experts," IEE Conference on Artificial Neural Networks, pp. 347-351, 1995.

[19] R. Polikar, L. Udpa, S. Udpa, V. Honavar, "Learn++: An incremental learning algorithm for multilayer neural networks," Proc. of IEEE Int. Conf. on Acoustics, Speech and Signal Proc., vol. 6, pp. 3414-3417, 2000.

[20] R. Polikar, engineering.rowan.edu/~polikar/RESEARCH/Learn++

[21] C.L. Blake and C.J. Merz, UCI Repository of machine learning databases at http://www.ics.uci.edu/~mlearn/MLRepository.html. Irvine, CA: University of California, Dept. of Information and Computer Science, 1998.